

자연어처리와 컴퓨터 비전 5-6장

📌 과제 유형	예습
📅 마감일	@2025년 5월 12일
☑ 발표	☐
☑ 완료	☐
📌 주차	10주차

5. 토큰화

- **자연어(Natural Language)**는 자연 언어라고도 부르며, 인공적으로 만들어진 프로그래밍 언어와 다르게 사람들이 쓰는 언어 활동을 위해 자연스럽게 만들어진 언어
- **자연어 처리(Natural Language Processing, NLP)**는 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술
- 자연어 처리는 인공지능의 하위 분야 중 하나로 컴퓨터가 인간과 유사한 방식으로 인간의 언어를 이해하고 처리하는 것이 주요 목표 중 하나이며, 인간 언어의 구조, 의미, 맥락을 분석하고 이해할 수 있는 알고리즘과 모델을 개발한다.
- 모델 개발을 위해 해결해야하는 문제
 - **모호성(Ambiguity)**: 인간의 언어는 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 되어 모호한 경우가 많다. 알고리즘은 이러한 다양한 의미를 이해하고 명확하게 구분할 수 있어야한다.
 - **가변성 (Variability)**: 인간의 언어는 다양한 사투리, 강세, 신조어, 작문 스타일로 인해 매우 가변적이다. 알고리즘은 이러한 가변성을 처리할 수 있어야하며 사용 중인 언어를 이해할 수 있어야한다.
 - **구조(Structure)**: 인간의 언어는 문장이나 구의 의미를 이해할 때 구문(Syntactic)을 파악하며 의미(Semantic)를 해석한다. 알고리즘은 문장의 구조와 문법적 요소를 이해하며 의미를 추론하거나 분석할 수 있어야한다.
- 위와 같은 문제를 이해하고 구분할 수 있는 모델을 만들려면 **우선 말뭉치(Corpus)를 일정한 단위인 토큰(Token)을 나눠야한다**. 말뭉치란 자연어 모델을 훈련하고 평가하는데 사용되는 대규모의 자연어이다.

- **말뭉치**는 뉴스 기사, 사용자 리뷰, 저널/칼럼 등에서 목적에 따라 구축되는 텍스트 데이터를 의미한다. 일련의 단어의 가능성을 예측하는 알고리즘인 언어 모델을 구축하고 평가하는데 자주 사용된다.
- **토큰**은 개별 단어나 문장 부호와 같은 텍스트를 의미하며 말뭉치보다 더 작은 단위이다. 토큰은 텍스트의 개별 단어, 구두점 또는 기타 의미 단위일 수 있다. 토큰으로 나누는 목적은 컴퓨터가 자연어를 이해할 수 있게 나누는 과정이다. 이러한 과정을 **토큰화(Tokenization)**라고 한다.
- **토큰나이저(Tokenizer)**는 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 소프트웨어를 의미하고, 이를 사용해 토큰화를 수행한다.
- 이 알고리즘은 텍스트에서 발생하는 다양한 단어와 구문을 식별하고 분석할 수 있게하므로 언어 모델링 또는 기계 번역과 같은 다양한 자연어 처리 작업에서 사용된다.
 - Ex. 입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'
 - 결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', '르', '수', '있', '다', '.']
- 일반적으로 토큰을 나누는 기준은 구축하려는 시스템이나 주어진 상황에 따라 다르고, 토큰을 어떻게 나누었느냐에 따라 시스템의 성능이나 처리 결과가 크게 달라지기도 한다. 토큰나이저를 구축하는 방법은 다음과 같다.
 - 공백 분할: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화한다.
 - 정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할한다.
 - 어휘 사전(Vocabulary) 적용: 사전에 정의된 단어 집합을 토큰으로 사용한다.
 - 머신 러닝 활용: 데이터셋을 기반으로 토큰화하는 방법을 학습한 머신 러닝을 적용한다.
- **어휘 사전(Vocabulary)**은 사전에 정의된 단어를 활용해 토큰나이저를 구축하는 방법이지만, 직접 어휘 사전을 구축하기 때문에 없는 단어나 토큰이 존재할 수 있다. 이러한 토큰을 **OOV(Out of Vocab)**이라 한다. OOV 문제를 고려해 토큰화하는 것이 중요하다.
 - 큰 어휘 사전을 구축하면 학습 비용 증대와 **차원의 저주 문제(Curse of Dimensionality)**에 빠질 수 있다.
 - 모든 토큰이나 단어를 벡터화하면 어휘 사전에 등장하는 토큰 개수만큼의 차원이 필요하고, 벡터값이 거의 모두 0의 값을 가지는 희소 데이터로 표현된다. 또한 어휘 사전에 등장하는 출현 빈도만 고려하기 때문에 문장에서 발생한 토큰들의 순서 관계를 잘 표현하지 못한다.

단어 및 글자 토큰화

- 토큰화는 데이터 전처리 과정, 텍스트 데이터를 구조적으로 분해하여 개별 토큰으로 나누는 작업을 의미한다. 이런 토큰화 가정은 정확한 분석을 위해 필수이며, 단어나 문장의 빈도수, 출현 패턴 등을 파악할 수 있다.
- 작은 단위로 분해된 텍스트 데이터는 컴퓨터가 이해하고 처리하기가 용이해 기계 번역, 문서 분류, 감성 분석 등 다양한 자연어 처리 작업에 활용할 수 있다.
- 텍스트 데이터를 단어나 글자 단위로 나누는 기법으로는 단어 토큰화와 글자 토큰화가 있다.

단어 토큰화 (Word Tokenization)

- 자연어 처리 분야에서 핵심적인 전처리 작업 중 하나로 텍스트 데이터를 의미있는 단위인 단어로 분리하는 작업이다.
- 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화가 수행된다. 단어 토큰화는 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용되며 가장 일반적인 토큰화 방법이다.

5.1 단어 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = review.split()  
print(tokenized)
```

출력 결과

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱',  
'최고!!']
```

- 문자열 데이터 형태는 `split` 메서드를 이용해 쉽게 토큰화할 수 있다. `split` 메서드는 주어진 구분자를 통해 문자열을 리스트 데이터로 나눠준다. 구분자를 입력하지 않으면 **공백(Whitespalce)**을 기준으로 나눈다.
- 나뉜 토큰을 보면 '최고!'와 '최고!!'를 비교해 보면 느낌표 하나의 차이로 다른 의미 단위로 나뉜다. 'cg.'도 마찬가지. 말뭉치 내에 마침표가 없는 'cg'라는 다른 토큰이 있다면, 이 두 토큰은 서로 다른 토큰으로 나뉜다.
- cg. 와 cg 가 비슷한 의미가 있는 것을 알고 있지만, 단어 토큰화를 통해 만들어진 단어 사전에서는 이 두 토큰은 다른 토큰이다. 즉, 'cg' 라는 토큰이 단어 사전 내에 있더라도, 'cg.', 'cg는', 'cg도' 등은 OOV가 된다. 이처럼 단어 토큰화는 한국어 접사, 문장 부호, 오타 혹은 띄어쓰기 오류 등에 취약하다.

글자 토큰화

- 글자 토큰화는 띄어쓰기뿐만 아니라 글자 단위로 문장을 나누는 방식으로, 비교적 작은 단어 사전을 구축할 수 있다는 장점이 있다. 작은 단어 사전을 사용하면 학습시 컴퓨터 자원을 아낄 수 있으며, 전체 말뭉치를 학습할 때 각 단어를 더 자주 학습할 수 있다.
- 글자 토큰화는 언어 모델링과 같은 시퀀스 예측 작업에서 활용된다. 예를 들어 다음 문장을 예측하는 언어 모델링에서 글자 토큰화는 유용한 방식이다.

5.2 단어 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = list(review)  
print(tokenized)
```

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', '시', '각', '적', ' ', '즐', '거', '움', '은', ' ', '최', '고', '!', ' ', '더', '불어', ' ', 'o', 's', 't', ' ', '는', ' ', '더', '더', '욱', ' ', '최', '고', '!', '!']
```

- 글자 토큰화는 리스트 형태로 변환하면 쉽게 수행할 수 있다. 단어 토큰화와 다르게 공백도 토큰으로 나뉜 것을 볼 수 있다. 영어의 경우 글자 토큰화를 진행하면 각 알파벳으로 나뉜다. 하지만 한글의 경우 하나의 글자는 여러 자음과 모음의 조합으로 이루어져 있다.
- 한글의 경우 자소 단위로 나눠서 자소 단위 토큰화를 수행한다. `jamo` 라이브러리(한글 문자 및 자모 작업을 위한 한글 음절 분해 및 합성)를 활용해 텍스트를 자소 단위로 분해해 토큰화를 수행한다.
- 자모 변환 함수

```
retval = jamo.h2j(hangul_string)
```

```
retval = jamo.j2hcj(jamo)
```

- 자모 변환 함수는 입력된 한글 문자열을 유니코드 U+1100~U+11FE 사이의 조합형 한글 자모로 변환하는 함수이다. 컴퓨터가 한글을 인코딩하는 방식은 크게 조합형과 완성형으로 나눌 수 있다.
 - 조합형: 글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글로 표현. 조합형 한글로 입력된 문자열은 초성, 중성, 종성으로 나뉜다.
 - 완성형: 조합된 글자 자체에 값을 부여해 인코딩
 - `h2j` 함수는 완성형 입력 ➡ 조합형 한글로 변환

- **j2hcj** 함수는 조합형 한글 문자열을 자소 단위로 나눠 반환하는 함수
- 이 두함수로 완성형 한글도 자소 단위로 나눌 수 있다.

5.3 자소 단위 토큰화

```
from jamo import h2j, j2hcj
```

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)
```

```
['ㅎ', 'ㄷ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅍ', ' ', 'ㄱ', 'ㅌ', 'ㅁ', 'ㅌ', 'ㄴ', ' ', 'ㅁ', 'ㅌ', 'ㄹ', 'ㄱ', 'ㅌ', 'ㅌ', 'ㅡ', 'ㅇ', 'ㅎ', 'ㅌ', 'ㄴ', ' ', 'ㄷ', 'ㄱ', ' ', ' ', 'ㅅ', 'ㅣ', 'ㄱ', 'ㅌ', 'ㄱ', 'ㅌ', 'ㅌ', ' ', 'ㅌ', 'ㅡ', 'ㄹ', 'ㄱ', 'ㅌ', 'ㅌ', 'ㅇ', 'ㅡ', 'ㅁ', 'ㅇ', 'ㅡ', 'ㄴ', ' ', 'ㅌ', 'ㅌ', 'ㅌ', 'ㄱ', 'ㅌ', '!', ' ', 'ㄷ', 'ㅌ', 'ㅁ', 'ㅌ', 'ㄹ', 'ㅇ', 'ㅌ', ' ', 'ㅇ', 'ㅌ', 'ㅌ', 'ㄴ', 'ㅡ', 'ㄴ', ' ', 'ㄷ', 'ㅌ', 'ㄷ', 'ㅌ', 'ㅇ', 'ㅌ', ' ', 'ㅌ', 'ㅌ', 'ㅌ', 'ㅌ', '!', '!']
```

- 자모 변환 함수와 한글 호환성 자모 변환 함수를 활용해 자소 단위로 분리. 글자 단위로 토큰화하면 단어 단위로 토큰화하는 것에 비해 비교적 적은 크기의 단어 사전을 구축할 수 있다.
- 앞서 단어 토큰화의 단점이었던 Cg 도 , cg 는 , Cg. 등에서 '도', '는', '.' 과 같은 접사와 문장 부호의 의미를 학습할 수 있다. 또 , 작은 크기의 단어 사전으로도 **OOV를 획기적으로 줄일 수 있다.**
- 하지만 개별 토큰은 아무 의미가 없으므로 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야한다. 토큰 조합 방식을 사용해 문장 생성이나 **개체명 인식(Name Entity Recognition)** 등을 구현할 경우, 다의어나 동음이의어가 많은 도메인에서 구별하는 것이 어려울 수 있다. 모델 입력 **시퀀스**의 길이가 길어질 수록 연산량이 증가한다는 단점이 있다.
 - 단어 토큰화 입력 시퀀스: 13
 - 글자 토큰화 입력 시퀀스: 53
 - 자소 단위 토큰화 입력 시퀀스: 101

형태소 토큰화 (Morpheme Tokenization)

- 텍스트를 형태소 단위로 나누는 토큰화 방법으로 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업

- 한국어와 같이 교착어(Agglutivative Language)인 언어에서 중요하게 수행된다. 한국어는 대부분의 언어와 달리 각 단어가 띄어쓰기로 구분되지 않는다. 한국어는 어근에 다양한 접사와 조사가 조합되어 하나의 낱말로 이루므로 각 형태소를 적절히 구분해 처리해야한다.
- 예를 들어 '그는 나에게 인사를 했다'라는 문장을 보면 '그는'은 '그' + '는'이 결합해 하나의 단어로 이루어져 있고, '나에게'는 '나'+ '에게'의 결합으로 이루어져 있다.
- '그', '-는', '나', '-에게' 등과 같이 실제로 의미를 가지고 있는 최소의 단위를 **형태소(Morpheme)**라고 한다. 형태소는 스스로 의미를 갖는 **자립 형태소(Free Morpheme)**와 스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용되는 **의존 형태소(Bound Morpheme)**으로 구분된다.
 - **자립 형태소:** 단어의 기본이 되는 형태소. 명사, 동사, 형용사와 같은 단어를 이루는 기본 단위
 - 그, 나, 인사
 - **의존 형태소:** 자립 형태소와 함께 조합되어 문장에서 특정한 역할을 수행하며, 조사, 어미, 접두사, 접미사 등이 해당된다.
 - -는, -에게, -를, 했-, -다

형태소 어휘 사전

- **형태소 어휘 사전(Morpheme Vocabulary)**은 자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중에서도 **각 단어의 형태소 정보를 포함**하는 사전. 단어가 어떤 형태소들의 조합으로 이루어져있는지에 대한 정보를 담고 있음
 - " 그는 나에게 인사를 했다"와 "나는 그에게 인사를 했다" 두 문장이 있고, 띄어쓰기를 활용해 토큰화를 수행한다면 어휘 사전은 ['그는', '나에게', '인사를', '했다', '나는', '그에게']로 구성된다.
 - 만약 말뭉치에 "그녀는 그에게 인사를 했다"라는 문장이 추가되면 모델은 '그는', '나는', '그녀는'과 같은 데이터를 의미 단위로 인식하지 못하고 학습을 진행한다. 이러한 문제를 방지하기 위해 형태소 단위로 어휘사전을 구축한다면 '-는', '-에게', '그', '나]와 같은 정보에서 ' 그녀' 의 토큰 정보만 새로 학습해 '그녀는' 이나 ' 그녀에게' 와 같은 어휘를 쉽게 학습할 수 있다.
 - 일반적으로 형태소 어휘 사전에는 각 형태소가 어떤 품사에 속하는지와 해당 품사의 뜻 등의 정보도 함께 제공된다.
- **품사 태깅(POS Tagging):** 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사(Part of Speech, POS)를 태깅하는 작업
 - 문맥을 고려하여 더 정확한 분석이 가능해짐

- "그는 나에게 인사를 했다"를 품사 태깅한다면, 그 (명사) +
는 (조사) + 나 (명사) + 에게 (조사) + 인사 (명사) + 를 (조사) + 했다 (동사)
로 태깅할 수 있다.

KoNLPy

- 한국어 자연어 처리를 위해 개발된 라이브러리로 명사 추출, 형태소 분석, 품사 태깅 등의 기능을 제공한다. 텍스트 데이터를 전처리하고 분석하기 위한 다양한 도구와 함수를 제공해 텍스트 마이닝, 감성 분석, 토픽 모델링 등 다양한 NLP 작업에서 사용
- 자바 개발 키트 기반으로 개발되어 자발 개발 키트가 설치되어 있어야 한다.
- Okt(Open Korean Text), 꼬꼬마 (Kkma), 코모란 (Komoran), 한나눔 (Hannanum), 메cab (Mecab) 등의 다양한 형태소 분석기를 지원한다.

5.4 Okt 토큰화

```
from konlpy.tag import Okt

okt = Okt()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = okt.nouns(sentence)
phrases = okt.phrases(sentence)
morphs = okt.morphs(sentence)
pos = okt.pos(sentence)

print("명사 추출:", nouns)
print("구 추출:", phrases)
print("형태소 추출:", morphs)
print("품사 태깅:", pos)
```

```
명사 추출 : ['무엇', '상상', '수', '사람', '무엇', '낼', '수']
구 추출 : ['무엇', '상상', '상상할 수', '상상할 수 있는 사람', '사람']
형태소 추출 : ['무엇', '이든', '상상', '할', '수', '있는', '사람', '은', '무엇', '이든', '만들어',
'낼', '수', '있다', '.']
품사 태깅 : [('무엇', 'Noun'), ('이든', 'Josa'), ('상상', 'Noun'), ('할', 'Verb'), ('수', 'Noun'),
('있는', 'Adjective'), ('사람', 'Noun'), ('은', 'Josa'), ('무엇', 'Noun'), ('이든', 'Josa'),
('만들어', 'Verb'), ('낼', 'Noun'), ('수', 'Noun'), ('있다', 'Adjective'), ('.', 'Punctuation')]
```

- Okt 객체는 문장을 입력 받아 명사, 구, 형태소, 품사 등의 정보를 추출하는 여러가지 메서드를 제공한다. Okt 에서 지원하는 대표적인 메서드는 **명사 추출 (okt.nouns)**, **구문 추출 (okt.phrases)**, **형태소 추출 (okt.morphs)**, **품사 태깅 (okt.pos)** 이다.
 - 명사 추출, 구문 추출, 형태소 추출은 입력된 문장에서 각각 명사, 어절 구 단위, 형태소만 추출해 리스트를 반환
 - 품사 태깅은 입력된 문장에서 각 단어에 대한 품사 정보를 추출하여 (형태사, 품사) 형태의 튜플 리스트를 반환

5.5 꼬꼬마 토큰화

```
from konlpy.tag import Kkma

kkma = Kkma()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = kkma.nouns(sentence)
sentences = kkma.sentences(sentence)
morphs = kkma.morphs(sentence)
pos = kkma.pos(sentence)

print("명사 추출:", nouns)
print("구 추출:", phrases)
print("형태소 추출:", morphs)
print("품사 태깅:", pos)
```

```
명사 추출 : ['무엇', '상상', '수', '사람', '무엇']
구문 추출 : ['무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다.']
형태소 추출 : ['무엇', '이', '든', '상상', '하', 'ㄹ', '수', '있', '는', '사람', '은', '무엇', '이', '든', '만들', '어', '내', 'ㄹ', '수', '있', '다', '.']
품사 태깅 : [('무엇', 'NNG'), ('이', 'VCP'), ('든', 'ECE'), ('상상', 'NNG'), ('하', 'XSV'), ('ㄹ', 'ETD'), ('수', 'NNB'), ('있', 'VV'), ('는', 'ETD'), ('사람', 'NNG'), ('은', 'JX'), ('무엇', 'NP'), ('이', 'VCP'), ('든', 'ECE'), ('만들', 'VV'), ('어', 'ECD'), ('내', 'VXV'), ('ㄹ', 'ETD'), ('수', 'NNB'), ('있', 'VV'), ('다', 'EFN'), ('.', 'SF')]
```

- 꼬꼬마는 Kkma 클래스를 통해 명사, 문장, 형태소, 품사를 추출할 수 있다. 꼬꼬마는 구문 추출 기능은 지원하지 않지만, **명사 추출(kkma.sentences)** 기능을 제공한다.

- 동일한 메서드라도 형태소 분석기의 특징에 따라 다른 결과가 나타난다.
 - 명사 추출의 경우 Okt 에서 반환하던 '넌' '수' 를 반환하지 않았으며 , 형태소 추출의 경우 '할' 을 더 분리해 '하' , 'f' 로 반환
 - 품사 태깅 역시 Okt는 총 19 개의 품사를 구분해 반환하는 한편, 꼬꼬마는 더 세분화해 56 개의 품사로 태깅한다. 태깅하는 품사의 수가 많으면 더 자세한 단위로 분석이 가능하지만, 품사 태깅에 소요되는 시간도 길어지며, 더 많은 품사로 분리해 모델의 성능이 저하될 수도 있다.
 - Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

- 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EF0	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

태그	품사	태그	품사
MAG	일반 부사	XPN	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	줄임표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 가운뎃점, 콜론, 빗금
JX	보조사	S0	붙임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

NLTK (Natural Language ToolKit)

- 자연어 처리를 위해 개발된 라이브러리로, 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능을 제공

- 주로 영어 자연어 처리를 위해 개발되었지만, 네덜란드어, 프랑스어, 독일어 등과 같은 다양한 언어의 자연어 처리를 위한 데이터와 모델을 제공
- NLTK 라이브러리를 활용해 토큰화나 품사 태깅 작업을 하기 위해서는 해당 작업을 수행할 수 있는 패키지나 모델을 다운로드 해야한다. Punkt 모델과 Averged Perceptron Tagger 모델을 활용해 토큰화 및 품사 태깅 작업을 수행한다.
 - 두 모델 모두 Treebank라는 대규모 영어 말뭉치를 기반으로 학습되었다. Punkt 모델은 통계 기반 모델이며, Averged Perceptron Tagger는 퍼셉트론을 기반으로 품사 태깅을 수행

5.6 패키지 및 모델 다운로드

```
import nltk

nltk.download("punkt_tab")
nltk.download("averaged_perceptron_tagger")
```

5.7 영문 토큰화

```
from nltk import tokenize

sentence = "Those who can imagine anything, can create the impossible."

word_tokens = tokenize.word_tokenize(sentence)
sent_tokens = tokenize.sent_tokenize(sentence)

print(word_tokens)
print(sent_tokens)
```

```
['Those', 'who', 'can', 'imagine', 'anything', ',', 'can', 'create', 'the', 'impossible', '.']
['Those who can imagine anything, can create the impossible.']
```

- 영문 토큰화는 사전에 설치한 **punkt** 모델을 기반으로 단어나 문장을 토큰화한다.
 - **단어 토큰나이저** (**word_tokenize**) 는 문장을 입력받아 공백을 기준으로 단어를 분리하고, 구두점 등을 처리해 각각의 단어를 추출해 리스트로 반환한다.
 - **문장 토큰나이저** (**sent_tokenize**) 는 문장을 입력받아 마침표 (.), 느낌표 (!), 물음표 (?) 등의 구두점을 기준으로 문장을 분리해 리스트로 반환
 - 영어는 한국어보다 토큰화하기 쉬움

5.8 영문 품사 태깅

```
nltk.download('averaged_perceptron_tagger_eng')  
# 추가로 다운로드 해야함
```

```
from nltk import tag  
from nltk import tokenize
```

```
sentence = "Those who can imagine anything, can create the impossible."
```

```
word_tokens = tokenize.word_tokenize(sentence)  
pos = tag.pos_tag(word_tokens)
```

```
print(pos)
```

```
[('Those', 'DT'), ('who', 'WP'), ('can', 'MD'), ('imagine', 'VB'), ('anything', 'NN'), (',', ','), ('can', 'MD'), ('create', 'VB'), ('the', 'DT'), ('impossible', 'JJ'), ('.', '.')]
```

- NLTK 라이브러리의 품사 태깅 메서드는 토큰화된 문장에서 품사 태깅을 수행한다. 품사 태깅을 수행하기 위해서는 토큰화된 단어들이 들어가야한다.
- Averaged Perceptron Tagger 모델은 총 35 개의 품사를 태깅할 수 있다.

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사

태그	품사	태그	품사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

spaCy

- Cython 기반으로 개발된 오픈 소스 라이브러리로, NLTK와 마찬가지로 자연어 처리 기능 제공
- NLTK와 주요한 차이점은 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반의 자연어 처리 라이브러리
 - NLTK 는 학습 목적으로 자연어 처리에 대한 다양한 알고리즘과 예제를 제공
 - spaCy 는 효율적인 처리 속도와 높은 정확도를 제공하는 것이 목표
- spaCy 는 GPU 가속을 비롯해 영어, 프랑스어, 한국어, 일본어 등을 비롯해 24개 이상의 언어로 사전 학습된 모델을 제공한다. 이 책에서는 SpaCy 라이브러리를 활용해 영어를 대상으로 토큰화할 예정이므로 영어로 사전 학습된 모델인 en_core_web_sm 을 설치

5.9 spaCy 품사 태깅

```
import spacy

nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything, can create the impossible."
doc = nlp(sentence)

for token in doc:
    print(f"[{token.pos_:5} - {token.pos_:3}] : {token.text}")
```

```

[PRON - DT ] : Those
[PRON - WP ] : who
[AUX - MD ] : can
[VERB - VB ] : imagine
[PRON - NN ] : anything
[PUNCT - , ] : ,
[AUX - MD ] : can
[VERB - VB ] : create
[DET - DT ] : the
[ADJ - JJ ] : impossible
[PUNCT - . ] : .

```

- spaCy는 사전 학습된 모델을 기반으로 처리하기 때문에 `load` 를 통해 모델을 설정할 수 있다.
- spaCy는 **객체 지향적**으로 구현되어 처리한 결과를 `doc` 객체에 저장한다. `doc` 객체는 다시 여러 `token` 객체로 이뤄져 있으며, 이 `token` 객체에 대한 정보를 기반으로 다양한 자연어 처리 수행
 - `nlp` 인스턴스에 문장 입력 ➡ `doc` 객체 반환 ➡ `token` 객체의 `tag_` 나 `pos_` 와 같은 속성에 접근해 값을 확인할 수 있다.
- `token` 객체에는 기본 품사 속성 (`pos_`), 세분화 품사 속성 (`tag_`), 원본 텍스트 데이터 (`text`), 토큰 사이의 공백을 포함하는 텍스트 데이터 (`text_with_ws`), 벡터 (`vector`), 벡터 노름 (`vector_norm`) 등의 속성이 포함되어 있다.

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	...	생략 부호
CCONJ	CC	접속사	PUNCT	“	여는 따옴표
DET	DT	관사, 한정사	PUNCT	”	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP\$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	\$	통화
DET	WP\$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VBN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
PART	TO	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

- 형태소 분석과 품사 태깅은 문장에서 의미를 갖는 최소 단위이므로 자연어 처리에서 매우 중요한 전처리 작업이다. 띄어쓰기와 맞춤법이 잘 지켜지지 않는 경우나 신조어나 외래어, 특정 도메인에서 사용되는 축약어 등은 완벽히 대응하기 어렵다. 그럼에도 불구하고 형태소 분석과 품사 태깅은 자연어 처리에서 필수적인 작업으로 이를 통해 단어의 의미를 파악하고 문장 구조를 이해할 수 있다.
- 형태소 분석과 품사 태깅은 주요 키워드를 추출하거나 문장의 긍정 / 부정 여부를 판단하는 언어 모델의 성능을 높이는 데도 중요한 역할을 한다. 이는 자연어 처리 분야에서 핵심적인 기술로 자리 잡고 있으며, 이를 활용하여 다양한 자연어 처리 응용 프로그램을 개발할 수 있다.

하위 단어 토큰화

- 자연어 처리에서 형태소 분석은 중요한 전처리 과정 중 하나이다. 형태소는 자연어의 최소 의미 단위이며, 대부분의 자연어가 형태소의 조합으로 이뤄져있다. 컴퓨터가 자연어를 인간이 이해하는 방식과 비슷하게 처리할 수 있게 하려면 형태소 단위의 토큰화가 효과적인 방법이다.
- 그러나 언어의 변동성, 맞춤법이나 띄어쓰기가 엄격하게 지켜지지 않은 경우, 신조어나 고유어 오타자 등으로 토큰화 성능이 감소할 수 있다.
 - Ex. 원문: 시보리도 짱짱해고 허리도 어병하지만구 죠하효
 - 결과: [시, 보리, 짱짱해고, 허리, 도, 어, 병하지만구, 죠, 하, 효]
 - '시보리'라는 외래어를 인식하지 못하고, 어병하다 역시 잘못 인식
- 또한 전문 용어나 고유어가 많은 데이터를 처리할 때 약점을 보인다. 즉, 형태소 분석기는 모르는 단어를 적절한 단어로 나누는 것에 취약하며, 이는 잠재적으로 어휘 사전의 크기를 크게 만들고 OOV에 대응하기 어렵게 만든다.
 - Ex. 원문 : 진짜 멋있네요 ! 돈썰날만 하네요.
 - 결과: [' 진짜', '멋있네요 ',' 돈썰날', ' 만' , ' 하네요 .']
 - '돈썰내다'라는 신조어를 잘 토큰화하지 못함. 어휘 사전의 크기를 크게 만들
- 현대 자연어 처리에서는 신조어의 발생, 오타자, 축약어 등을 고려해야하기 때문에 분석할 단어의 양이 많아져 어려움을 겪는다. 이를 해결하기 위해 **하위 단어 토큰화 (Subword Tokenization)**를 사용한다.
- 하위 단어 토큰화라나 하나의 단어가 빈번하게 사용되는 하위 단어 (Subword)의 조합으로 나누어 토큰화하는 방법이다.
 - Ex. 'Reinforcement' → 'Rein' + 'force' + 'ment'
- 하위 단어 토큰화를 적용하면 단어의 길이를 줄일 수 있어서 처리 속도가 빨라질 뿐만 아니라, OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제도 완화할 수 있다.
- 바이트 페어 인코딩, 워드 피스, 유니그램 모델 등

바이트 페어 인코딩 (Byte Pair Encoding, BPE)

- 다이어그램 코딩이라고도 한다. 텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합으로 찾아 부호화하는 압축 알고리즘으로 초기에는 데이터 압축을 위해 개발되었으나, 자연어 처리 분야에서 하위 단어 토큰화 방법으로 사용됨
- 이 알고리즘은 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복하며 이 과정에서 자주 등장하는 단어는 하나의 토큰으로 토큰화되고, 덜 등장하는 단어는 여러 토큰의 조합으로 표현됨

- Ex. abracadbra → ab: A → AracadAra → ra: B → ABcadAB → AB: C → CcadC
- 바이트 페어 인코딩은 입력 데이터에서 가장 많이 등장한 글자의 빈도수를 측정하고, 가장 빈도수가 높은 글자쌍을 탐색하고 치환한다.
- 토큰나이저로써 바이트 페어 인코딩은 자주 등장하는 글자쌍을 찾아 치환하는 대신 어휘 사전에 추가한다.
- 말뭉치에서 바이트 페어 인코딩을 적용해보자. 말뭉치에서 각 단어가 등장한 빈도를 계산해 다음과 같은 빈도 사전과 어휘 사전을 만들었다고 가정해보자.
 - 빈도 사전: ('low', 5), ('lower', 2), ('newest', 6), ('widest', 3)
 - 어휘 사전: ['low', 'lower', 'newest', 'widest']
- 빈도 사전을 바이트 페어 인코딩으로 재구성한다고 가정해보자. 이 알고리즘을 적용하기 위해 빈도 사전 내 모든 단어를 글자 단위로 나눈다.
 - 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'l', 'l', 'n', 'o', 'r', 's', 't', 'w']
- 빈도 사전을 기준으로 가장 자주 등장한 글자 쌍을 찾는다. 빈도 사전에서 'e', 's' 쌍이 'newest'에서 6번, 'widest'에서 3번 등장해 총 9 번으로 가장 많이 등장. 그러므로 빈도 사전에서 'e'와 's'를 'es'로 병합하고 어휘 사전에 추가한다.
 - 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'es', 't', 6), ('w', 'i', 'd', 'es', 't', 3)
 - 어휘 사전: ['d', 'e', 'l', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es']
- 동일한 과정을 다시 반복. 가장 빈도수가 높은 'es'와 't' 쌍을 'est'로 병합하고 어휘 사전에 추가한다.
 - 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
 - 어휘 사전: ['d', 'e', 'l', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est']
- 이러한 과정을 10번 반복했다고 가정하면 다음과 같은 빈도 사전과 어휘 사전이 생성된다.

- 빈도 사전: ('low', 5), ('low', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est', 'lo', 'low', 'ne', 'new', 'newest', 'wi', 'wid', 'widest']
- BPE 알고리즘을 사용해 말뭉치에서 자주 등장하는 글자 쌍을 찾아 어휘 사전을 구축했다. 'newest', 'wider', 'lowest'와 같이 기존 말뭉치에 등장하지 않았던 단어가 입력되더라도 OOV로 처리되지 않고 기존 어휘 사전을 참고해 'new e r', 'wid e r', 'low est'와 같이 토큰화할 수 있다.
- 대량의 말뭉치를 사용해 토크나이저를 학습한다면 효율적 학습을 위해 이미 구현되어 있는 토크나이저 라이브러리를 사용한다.

센텐스피스

- 이번 절에서는 **Sentencepiece** 라이브러리와 **Korpora** 라이브러리를 활용해 토크나이저를 학습해본다.
 - 센텐스피스 라이브러리는 구글에서 개발한 오픈 소스 하위 단어 토크나이저 라이브러리이다. 바이트 페어 인코딩과 유사한 알고리즘을 사용해 입력 데이터를 토큰화하고 단어 사전을 생성. 워드피스, 유니코드 기반의 다양한 알고리즘을 지원하며 사용자가 직접 설정할 수 있는 하이퍼 파라미터들을 제공해 세밀한 토크나이징 기능을 제공한다.
 - 코포라 라이브러리는 국립국어원이나 AI Hub에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리다.

토크나이저 모델 학습

- 2017년 8월부터 2019년 3월까지 청와대 청원 게시판에 올라온 청원 말뭉치를 가져온다.

5.10 청와대 청원 데이터 다운로드

```
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
dataset = corpus.train
petition = dataset[0]

print("청원 시작일:", petition.begin)
print("청원 종료일:", petition.end)
print("청원 동의수:", petition.num_agree)
print("청원 범주:", petition.category)
```

```
print("청원 제목:", petition.title)
print("청원 본문:", petition.text[:30])
```

청원 시작일 : 2017-08-25

청원 종료일 : 2017-09-24

청원 동의 수 : 88

청원 범주 : 육아/교육

청원 제목 : 학교는 인력센터, 취업센터가 아닙니다. 정말 간곡히 부탁드립니다.

청원 본문 : 안녕하세요. 현재 사대, 교대 등 교원양성학교들의 예비

- 코포라 라이브러리는 말뭉치 불러오기(Korpora.load)를 통해 말뭉치 데이터를 다운로드할 수 있다.
- corpus 는 총 433.631 개의 청원 데이터가 저장돼 있으며 , train 속성으로 학습 데이터를 가지고 올 수있다. 첫 번째 청원 데이터는 dataset[0]으로 불러올 수 있으며. petition에 청원 시작일 , 동의 수 , 제목 등의 정보가 포함돼 있다.

5.11 학습 데이터셋 생성

```
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
petitions = corpus.get_all_texts()
with open('datasets/corpus.txt', 'w', encoding = 'utf-8') as f:
    for petition in petitions:
        f.write(petition + '\n')
```

- corpus 의 get_all_texts 메서드로 본문 데이터셋을 한 번에 불러올 수 있다. 청원 데이터를 하나의 텍스트 파일로 저장한다.

5.12 토큰나이저 모델 학습

- corpus.txt 파일을 활용해 SentencePieceTrainer 로 토큰나이저 모델 학습을 진행한다.

```
from sentencepiece import SentencePieceTrainer

SentencePieceTrainer.Train("--input=datasets/corpus.txt --model_prefix=peti")
```

- 센텐스 피스 라이브러는 SentencePieceTrainer 클래스의 Train 메서드로 토큰나이저 모델을 학습할 수 있다. 센텐스피스 는 문자열 입력을 통해 인자들을 전달받는다. 센텐스

피스 라이브러리의 주요한 하이퍼 파라미터는 다음과 같다.

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토크나이저가 다룰 수 있는 글자의 비율
model_type	토크나이저 알고리즘 ■ unigram ■ bpe ■ char ■ word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

- 센텐스피스 라이브러리에서 지원하는 모든 라이브러리는 깃허브에서 확인 가능

<https://github.com/google/sentencepiece/blob/master/doc/options.md>

- 토크나이저 모델 학습이 완료되면 `petition_bpe.model` 파일과 `petition_bpe.vocab` 파일이 생성된다. `model` 파일은 학습된 토크나이저가 저장된 파일이며, `vocab` 파일은 어휘 사전이 저장된 파일이다.
- 센텐스피스 라이브러리는 띄어쓰기나 공백도 특수문자로 취급하여 `_(underscore)`로 공백 표현
 - Hello world 라는 문장은 `Hello_world` 로 표현되며 토큰화하면 `'_Hello + _World'` 로 토큰화된다.

5.13 바이트 페어 인코딩 토큰화

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
```

```

tokenizer.load('petition_bpe.model')

sentence = "안녕하세요, 토큰라이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰라이저를 사용할 수 있답니"]

tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)

print("단일 문장 토큰화:", tokenized_sentence)
print("여러 문장 토큰화:", tokenized_sentences)

encoded_sentence = tokenizer.encode_as_ids(sentence)
encoded_sentences = tokenizer.encode_as_ids(sentences)
print("단일 문장 토큰화:", encoded_sentence)
print("여러 문장 토큰화:", encoded_sentences)

decode_ids = tokenizer.decode_ids(encoded_sentences)
decode_pieces = tokenizer.decode_pieces(encoded_sentences)
print("정수 인코딩에서 문장 변환:", decode_ids)
print("하위 단어 토큰에서 문장 변환:", decode_pieces)

```

```

단일 문장 토큰화 : ['_안녕하세요', ',', '_토', '크', '나', '이', '저', '가', '_잘', '_학', '습',
'되었', '군요', '!']
여러 문장 토큰화 : [['_이렇게', '_입', '력', '값을', '_리', '스트', '로', '_받아서'], ['_쉽게',
'_토', '크', '나', '이', '저', '를', '_사용할', '_수', '_있', '답니다']]
단일 문장 정수 인코딩 : [664, 6553, 991, 6880, 6544, 6513, 6590, 6523, 159, 110, 6554, 868, 782,
6648]
여러 문장 정수 인코딩 : [[370, 180, 6677, 4427, 1768, 1610, 6527, 4157], [1677, 991, 6880, 6544,
6513, 6590, 6536, 5848, 18, 5, 2633]]
정수 인코딩에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수
있답니다']
하위 단어 토큰에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수
있답니다']

```

- 센텐스피스 토큰라이저 모델은 `SentencePieceProcessor` 클래스를 통해 학습된 모델을 불러올 수 있다. 토큰라이저 모델 불러오기 메서드(`tokenizer.load`)를 통해 `petition_bpe.model` 모델을 불러온다.

- `encode_as_pieces` 메서드는 문장을 토큰화하며, `encode_as_ids` 메서드는 토큰을 정수로 인코딩해 제공한다. 이 정수 데이터는 어휘 사전의 토큰에 매핑된 ID 값을 의미한다. 이 ID 값을 활용해 자연어 처리 모델을 구축한다.
- 토큰나이저 모델이나 자연어 처리 모델에서 나온 정수는 `decode_ids` 메서드나 `decode_pieces` 메서드를 통해 문자열 데이터로 변환 가능

5.14 어휘 사전 불러오기

```
from sentencepiece import SentencePieceProcessor
```

```
tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")
```

```
vocab = {idx: tokenizer.id_to_piece(idx) for idx in range(tokenizer.get_piece_size())}
print(list(vocab.items())[:5])
print("vocab size:", len(vocab))
```

```
[(0, '<unk>'), (1, '<s>'), (2, '</s>'), (3, '니다'), (4, '_이')]
vocab size : 8000
```

- `get_piece_size` 메서드는 센텐스피스 모델에서 생성된 하위 단어의 개수를 반환하며, `id_to_piece` 메서드는 정수값을 하위 단어로 변환하는 메서드다. 그러므로 하위 단어의 개수만큼 반복해 하위 단어 딕셔너리를 구성한다.
- 토큰 딕셔너리를 출력하면 `<unk>`, `<s>`, `</s>`가 존재. `<unk>`: unknown의 약자로 OOV 발생 시 매핑되는 토큰이며, `<s>`와 `</s>`는 문장의 시작 지점과 종료 지점을 표시하는 토큰
- 어휘 사전의 크기는 모델 학습시 설정한 8,000으로 학습됐다. 센텐스피스 라이브러리는 설정값에 따라 학습 방법이나 어휘 사전 크기가 달라질 수 있다.

워드 피스

- 워드 피스 토큰나이저는 바이트 페어 인코딩 토큰나이저와 유사한 방법으로 학습되지만, 빈도 기반이 아닌 확률 기반으로 글자쌍을 병합한다.
- 워드 피스는 학습 과정에서 확률적인 정보를 사용한다. 모델이 새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택한다.

- 이렇게 선택된 하위 단어는 이후에 더 높은 확률로 선택될 가능성이 높으며, 이를 통해 모델이 좀 더 정확한 하위 단어로 분리할 수 있다.

수식 5.1 글자 쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x)f(y)}$$

- f: frequency
- x, y: 병합하려는 하위 단어
- f(x, y): x와 y가 조합된 글자쌍의 빈도
- score: x와 y를 병합하는 것이 적절한지를 판단하기 위한 점수
- 워드 피스도 바이트 페어 인코딩과 마찬가지로 빈도 사전 내의 모든 단어를 글자 단위로 나눈다.

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

- 가장 빈번하게 등장하는 쌍은 'e'와 's'로 9번 등장한다. 'e'는 17번, 's'는 9번 등장하므로 점수는 $9/(17*9) = 0.06$ 이 된다. 'i'와 'd' 쌍은 3번 밖에 등장하지 않았지만 'i'와 'd'가 각각 3번씩 등장하므로 점수는 $3/(3*3) = 0.33$ 이 된다. 따라서 'i'와 'd' 쌍을 병합한다.

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id']

- 동일한 과정을 반복해 각 글자쌍에 대한 점수를 계산한다. 다음으로는 'l'과 'o' 글자쌍을 병합한다.

- 빈도 사전: ('lo', 'w', 5), ('lo', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id', 'lo']

토큰나이저스

- 토큰나이저스 라이브러리의 워드피스 API를 이용하면 쉽고 빠르게 토큰나이저를 구현하고 학습할 수 있다. 허깅 페이스의 **토큰나이저스(Tokenizers)** 라이브러리 사용

- 토큰라이저스 라이브러리는 **정규화(Normalization)**와 **사전 토큰화(Pre-tokenization)**를 제공한다.
- 정규화는 일관된 형식으로 텍스트를 표준화하고 모호한 경우를 방지하기 위해 일부 문자를 대체하거나 제거하는 등의 작업을 수행한다. 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등
- 사전 토큰화는 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나누는 기능을 제공한다. 공백 혹은 구두점을 기준으로 입력 문장을 나눠 텍스트 데이터를 효율적으로 처리하고 모델의 성능을 향상시킬 수 있다.

5.15 워드피스 토큰라이저 학습

```
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(WordPiece()) # 워드피스 모델 호출
tokenizer.normalizer = Sequence([NFD(), Lowercase()]) # 정규화 방식 설정 (NFD, Lowercase)
tokenizer.pre_tokenizer = Whitespace() # 사전 토큰화 방식 설정 (공백, 구두점 기준)

tokenizer.train(['datasets/corpus.txt']) # 훈련 메서드를 통해 학습에 사용하려는 데이터셋 경로 전달
tokenizer.save(['models/petition_wordpiece.json']) # 학습 완료 -> 학습 결과 저장
```

- 토큰라이저스 라이브러리의 토큰라이저(`Tokenizer`)로 워드피스(`WordPiece`) 모델을 불러온다. 모델을 불러왔다면 정규화 방식과 사전 토큰화 방식을 설정한다.
- **정규화 방식**은 `normalizers` 모듈에 포함된 클래스를 불러와 시퀀스(`Sequence`) 형식으로 인스턴스를 전달한다. 이번 예제에서 적용한 정규화 방식은 NFD unicode normalization(`NFD`), Lowercase transformation(`Lowercase`)를 사용한다.
- 사전 토큰화 방식도 `pre_tokenizers` 모듈에 포함된 클래스를 불러와 적용한다. 이번 예제에서 적용한 사전 토큰화 방식은 공백과 구두점을 기준으로 분리한다.
- `train` method를 통해 학습에 사용하려는 데이터셋 경로 전달. 학습이 완료되었다면 `save` method로 학습 결과 저장
- 정규화 클래스
- 사전 토큰화 클래스

이름	설명
NFD()	NFD 유니코드 정규화
NFKD()	NFKD 유니코드 정규화
NFC()	NFC 유니코드 정규화
NFKC()	NFKC 유니코드 정규화
Lowercase()	입력 문장의 모든 대문자를 소문자로 변환
Strip()	입력 문장의 양 끝에 있는 공백 제거
StripAccents()	모든 액센트 기호 제거
Replace()	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
BertNormalizer()	BERT 모델에 사용된 정규화 적용
Sequence()	여러 개의 정규화 모듈을 병합해 순서대로 수행

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
CharBelieverSplit()	문자 기준으로 문자열을 나눈다.
Digits()	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
Punctuation()	구두점을 기준으로 입력 문자열을 나눈다.
Metaspace(replacement=" ",)	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다. * 기본값: „\u2580“
Whitespace()	단어 경계공백과 구두점을 기준으로 사전 토큰화를 진행한다. * \w+["'ws]+ 정규 표현식을 적용한다.
WhitespaceSplit()	공백 문자를 기준으로 입력 문자열을 나눈다.
Split(pattern, behavior)	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. - behavior - removed: 삭제 - isolated: 개별 토큰 처리 - merged_with_previous: 이전 문자열과 결합해 토큰 처리 - merged_with_next: 다음 문자열과 결합해 토큰 처리 - contiguous: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리

5.16 워드피스 토큰화

```

from tokenizers import Tokenizer
from tokenizers.decoders import WordPiece as WordPieceDecoder

tokenizer = Tokenizer.from_file('models/petition_wordpiece.json')
tokenizer.decoder = WordPieceDecoder()

sentence = "안녕하세요. 워드피스 토큰라이저입니다."
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰라이저를 사용할 수 있답니"]

encoded_sentence = tokenizer.encode(sentence)
encoded_sentences = tokenizer.encode_batch(sentences)

print("인코더 형식:", type(encoded_sentence))

print("단일 문장 토큰화:", encoded_sentence.tokens)
print("여러 문장 토큰화:", [enc.tokens for enc in encoded_sentences])

print("단일 문장 정수 인코딩:", encoded_sentence.ids)
print("여러 문장 정수 인코딩:", [enc.ids for enc in encoded_sentences])

print("정수 인코딩에서 문장 변환:", tokenizer.decode(encoded_sentence.ids))

```

```

인코더 형식 : <class 'tokenizers.Encoding'>
단일 문장 토큰화 : ['안녕하세요', '.', '워', '##드', '##피', '##스', '토', '##크', '##나이', '##저', '##입니다', '.']
여러 문장 토큰화 : [['입력', '##값을', '리스트', '##로', '전달', '##해'], ['쉽게', '토', '##크', '##나이', '##저', '##를', '사용할', '수', '있습니다', '.']]
단일 문장 정수 인코딩 : [8760, 13, 10398, 7638, 7955, 7512, 8693, 8415, 16269, 7536, 7598, 13]
여러 문장 정수 인코딩 : [[19643, 13834, 28119, 7495, 11589, 7502], [9739, 8693, 8415, 16269, 7536, 7510, 14129, 7562, 7698]]
정수 인코딩에서 문장 변환 : 안녕하세요. 워드피스 토큰나이저입니다.

```

- 워드피스 토큰화를 수행하기 위해 모델 결과가 저장된 `petition_wordpiece.json` 파일을 불러와 `Tokenizer` 객체를 생성
- `WordPieceDecoder()` 를 사용해 `Tokenizer`의 디코더를 워드피스 디코더로 설정
- 인코딩(`encode`) 메서드로 문장을 토큰화할 수 있으며, 인코딩 배치(`encode_batch`) 메서드로 여러 문장을 한 번에 토큰화할 수 있다. 인코딩 메서드와 인코딩 배치 메서드 모두 워드피스 토큰나이저를 통해 문장을 토큰화하고 각 토큰의 색인 번호 반환
- 토큰화된 데이터는 토큰(tokens) 속성을 통해 값을 확인할 수 있으며, 토큰 정수 (ids) 속성으로 인코딩된 문장의 ID 값을 출력할 수 있다.
- 정수를 다시 문장으로 변환하는 경우 디코딩(`decode`) 메서드를 통해 정수 인코딩된 결과를 다시 문장으로 디코딩해 출력할 수 있다.
- 이 책에서 소개된 알고리즘 이외에도 유니코드 단위가 아닌 바이트 단위에서 토큰화하는 바이트 수준 바이트 페어 인코딩 (Byte-level Byte-Pair-Encoding, BBPE) 이나 크기가 큰 어휘 사전에서 덜 필요한 토큰을 제거하며 학습하는 유니그램 (Unigram) 등이 있다.

임베딩

- 앞서 학습한 토큰화만으로는 모델을 학습할 수 없다. 컴퓨터는 텍스트 자체를 이해할 수 없으므로 텍스트를 숫자로 변환하는 **텍스트 벡터화 (Text Vectorization)** 과정이 필요하다.
- 텍스트 벡터화란 텍스트를 숫자로 변환하는 과정으로, **원핫인코딩 (One-Hot Encoding)**과 **빈도 벡터화 (Count Vectorization)**등이 있다.
- 원핫인코딩
 - 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑한 후, 해당 색인 위치를 1로 표시하고 나머지 위치는 모두 0으로 표시하는 방식
 - 같은 단어가 여러번 등장하더라도 1과 0으로 표현
 - Ex. 'I like apples' 문장과 'I like bananas' 문장

- 띄어쓰기 기준으로 토큰화하고 단어를 고유한 색인 값으로 매핑

색인	0	1	2	3
토큰	I	like	apples	bananas

- 원-핫인코딩

- I like apples: [1, 1, 1, 0]
- I like bananas: [1, 1, 0, 1]

- 빈도 벡터화
 - 문서에서 단어의 빈도수를 세어 **해당 단어의 빈도를 벡터로 표현**
 - apples라는 단어가 총 4번 등장하면 해당 단어의 벡터값은 4가 됨
- 이러한 방법들은 단어나 문장을 벡터 형태로 변환하기 쉽고 간단하지만, 벡터의 **희소성 (Sparsity)**이 크다는 단점이 있다.
 - 말뭉치 내에 존재하는 토큰 개수만큼 벡터의 차원을 가져야 하지만, 입력 문장 내에 존재하는 토큰의 수는 그에 비해 현저히 적기 때문에 컴퓨팅 비용의 증가와 차원의 저주 같은 문제를 겪을 수 있다.
 - 또한 텍스트의 벡터가 입력 텍스트의 의미를 내포하고 있지 않으므로 두 문장이 의미적으로 유사하다고 하더라도 벡터는 유사하게 나타나지 않을 수 있음
 - Ex. 'i scream for ice cream'과 'ice cream for i scream'에 원-핫 인코딩이나 빈도 벡터화를 적용하면 둘 다 [1, 1, 1, 1, 1] 벡터로 반환됨
- 이러한 문제를 해결하기 위해 워드 투 벡터(Word2Vec)나 패스트 텍스트(fastText) 등과 같이 단어의 의미를 학습해 표현하는 **워드 임베딩 기법(Word Embedding)**을 사용
 - **워드 임베딩 기법은 단어를 고정된 길이의 실수 벡터로 표현하는 방법**으로, 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론
 - 워드 임베딩은 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어렵다는 단점이 있어 인공 신경망을 활용해 **동적 임베딩(Dynamic Embedding)** 기법을 사용

언어 모델

- **언어 모델 (Language Model)**이란 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델을 의미한다. 이를 위해 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행한다.

- 언어 모델은 자동 번역, 음성 인식, 텍스트 요약 등 다양한 자연어 처리 분야에서 활용됨

$P(\text{안녕하세요} \mid \text{안녕하세요}) = 0.081$	...	$P(\text{서울특별시}) = 0.59$
Input : 안녕하세요		Input : 대한민국 ... 강남구
$P(\text{아이오. 괜찮습니다.}) = 0.000001$		$P(\text{아이스 아메리카노}) = 0.000001$

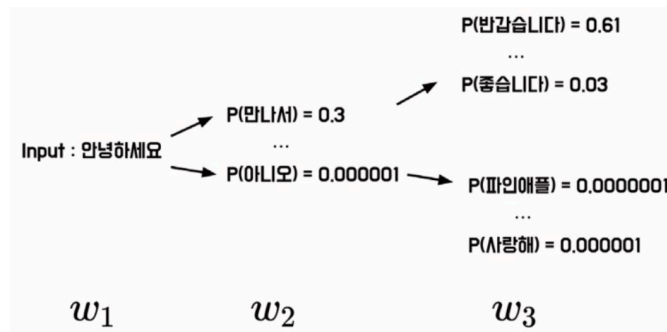
- '안녕하세요'라는 문장 뒤에 어떤 문장이 어떤 확률로 등장할지 계산하는 것은 어렵다. 이는 비지도 학습 문제로 모델 스스로 문장 등장 확률을 계산해야한다. 또한 '안녕하세요' 다음에 올 수 있는 문장은 사실상 무한에 가깝기 때문에 문장 단위로 확률을 계산하는 것은 불가능하다.
- 문장 전체를 예측하는 방법 대신 하나의 토큰 단위로 예측하는 방법인 **자기회귀 모델**이 고안되었다.

자기회귀 언어 모델 (Autoregressive Language Model)

- 입력된 문장들의 조건부 확률을 이용해 **다음에 올 단어를 예측**한다. 즉, 언어 모델에서 조건부 확률은 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 계산하는 것
- 이를 위해 이전에 등장한 모든 토큰의 정보를 고려하며, 문장의 문맥 정보를 파악하여 다음 단어를 생성한다. 다음 단어는 다시 이전 단어를 기반으로 예측이 이루어지며, 이 과정이 반복된다. 수식으로 표현하면 다음과 같다. (**수식 6.1**)

$$P(w_t \mid w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

- 언어 모델에서 조건부 확률을 계산하기 위해 이전에 등장한 단어 시퀀스($w_1, w_2, \dots, w_{t-1}$)를 기반으로 다음 단어의 확률을 계산한다. 수식 6.1에 **조건부 확률의 연쇄 법칙(Chain rule for conditional probability)**을 적용하면 **수식 6.2**와 같다.
 - 문장 전체의 확률은 첫 번째 단어의 확률 $P(w_1)$ 과 각 단어가 이전 단어들의 조건부 확률에 따라 발생할 확률의 곱으로 나타낼 수 있음. 이를 통해 언어 모델은 문장 전체의 확률을 계산하고, 그것을 이용해 다음 단어를 예측함
- '안녕하세요' 다음에 등장할 수 있는 문장을 조건부 확률의 연쇄 법칙 방법으로 시각화하면 다음과 같다.



- 자기회귀 언어 모델에서는 각 시점에서 다음에 올 토큰을 예측하는 것이 중요하다. 이를 위해 입력 토큰 w_1 을 바탕으로 모델은 다음 토큰 w_2 가 등장할 확률 $P(w_2|w_1)$ 을 예측한다. 그다음 모델의 출력값 w_2 가 다시 입력값이 되어 모델은 확률 $P(w_3|w_1, w_2)$ 을 예측한다.
 - 이러한 방식으로 모델의 출력값이 다시 모델의 입력값으로 사용되는 특징 때문에 자기회귀라는 이름이 붙었다. 자기 회귀 언어 모델은 시점 별로 다음에 올 토큰을 예측하는 것이므로, 토큰 분류 문제로 정의할 수 있음
 - 자기 회귀 모델은 이전에 등장한 모든 토큰 정보를 활용해 입력된 문장의 문맥 정보를 파악하고 다음 토큰을 예측한다. 그러므로 언어 모델은 문장 전체의 확률을 계산하고, 이를 이용해 다음 단어를 예측한다.

통계적 언어 모델(Statistical Language Model)

- 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석한다. 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률을 예측한다.
- 일반적으로 통계적 언어 모델은 **Markov Chain**을 이용해 구현된다. 마르코프 체인은 빈도 기반의조건부 확률 모델 중 하나로 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측한다.
- 빈도 기반의 조건부 확률 모델에서는 주어진 데이터에서 각 변수가 발생한 빈도수를 기반으로 확률을 계산한다. 예를 들어 말뭉치에 '안녕하세요'라는 문장이 1,000번 등장하고 이어서 '안녕하세요 만나서'가 700번, '안녕하세요 반갑습니다'가 100번 등장했다고 가정하면 빈도의 기반 조건부 확률 수식은 다음과 같이 표현된다. **(수식 6.3)**

$$P(\text{만나서}|\text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다}|\text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

- 이 방법은 단어의 순서와 빈도에만 기초해 문장의 확률을 예측하므로 문맥을 제대로 파악하지 못하면 불완전하거나 부적절한 결과를 생성할 수 있다.
- 또 **한 번도 등장한 적 없는 단어나 문장에 대해서는 정확한 확률을 예측하기 어렵다.** 이렇게 관측한 적 없는 데이터를 예측하지 못하는 문제를 **데이터 희소성 (Data sparsity)** 문제라고 한다.
- 하지만 통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를 생성하므로, 이를 이용해 새로운 문장을 생성할 수 있으며, 다양한 종류의 텍스트 데이터를 학습할 수 있다.
- 대규모 자연어 데이터를 처리하는데 효과적이며, 딥러닝 등의 인공지능 기술이 발전하면서 더욱 강력한 모델을 구현할 수 있게 되었다.
- 최근 연구되는 자연어 처리 기법은 언어 모델을 활용해 가중치를 사전학습함. 예를 들어 트랜스포머 모델에 기반한 언어 모델인 GPT(Generative Pre-trained Transformer)나 BERT(Bidirectional Encoder Representations from Transformer)는 다음과 같은 문장 생성 기법을 이용해 모델을 사전 학습한다.

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

N-gram

- 가장 기초적인 통계적 언어 모델은 N-gram 모델이다. N-gram 모델은 텍스트에서 N 개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정한다.
- N-gram 모델은 입력 텍스트를 하나의 토큰 단위로 분석하지 않고 N개의 토큰을 묶어서 분석한다. 이때, 연속된 N 개의 단어를 하나의 단위로 취급하여 추론하는 모델이며, N이 1일 때는 **유니그램(Unigram)**, N이 2일 때는 **바이그램(Bigram)**, N이 3일 때는 **트라이그램(Trigram)**으로 부른다. $N \geq 4$ 일 때는 N-gram이라 한다.

Unigram : <u>안녕하세요</u> 만나서 진심으로 반가워요	안녕하세요, 만나서, 진심으로, 반가워요
Bigram : <u>안녕하세요</u> <u>만나서</u> 진심으로 반가워요	안녕하세요 만나서, 만나서 진심으로, 진심으로 반가워요
Trigram : <u>안녕하세요</u> <u>만나서</u> <u>진심으로</u> 반가워요	안녕하세요 만나서 진심으로, 만나서 진심으로 반가워요

- N-gram 언어 모델은 모든 토큰을 사용하지 않고 N-1개의 토큰만을 고려해 확률을 계산한다. N-gram 모델에서 t 번째 토큰의 조건부 확률을 계산하는 수식은 다음과 같다. **(수식 6.4)**

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

- w_t 는 예측하려는 단어, w_{t-1} 부터 w_{t-N+1} 까지는 예측에 사용되는 이전 단어들을 의미한다. 이전 단어들의 개수를 결정하는 N 값을 조정해 N-gram 모델의 성능을 조절할 수 있다.

6.1 N-gram

```
import nltk

def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
trigram = ngrams(sentence, 3)

print(unigram)
print(bigram)
print(trigram)

unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams(sentence.split(), 2)
trigram = nltk.ngrams(sentence.split(), 3)

print("=====")
```

```
print(list(unigram))
print(list(bigram))
print(list(trigram))
```

```
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
```

- 통계적 언어 모델은 상대적으로 구현이 쉽다. 파이썬으로 구현한 N-gram과 NLTK 라이브러리에서 지원하는 N-gram 함수는 동일한 출력값을 반환한다.
- N-gram은 작은 규모의 데이터세트에서 연속된 문자열 패턴을 분석하는데 큰 효과를 보이며, 관용적 표현 분석에서도 활용된다.
 - 예를 들어 '입이 무겁다'는 '비밀을 잘 지킨다'라는 뜻이며 자주 등장하는 연속된 단어나 구를 추출하고, 이를 분석함으로써 관용적 표현을 파악할 수 있다.
- N-gram은 시퀀스에서 연속된 n 개의 단어를 추출하므로 단어의 순서가 중요한 자연어 처리 작업 및 문자열 패턴 분석에 활용된다.

TF-IDF (Term Frequency-Inverse Document Frequency)

- 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법으로, 문서 내에서 단어의 중요도를 평가하는데 사용되는 통계적인 가중치를 의미한다. TF-IDF는 **BoW(Bag-of-Words)**에 가중치를 부여하는 방법이다.
- BoW는 문서나 문장을 단어의 집합으로 표현하는 방법으로, 문서나 문장에 등장하는 단어의 중복을 허용해 빈도를 기록한다. 원-핫 인코딩은 단어의 등장 여부를 판별해 0과 1로 표현하는 방식이지만, **BoW는 등장 빈도를 고려해 표현한다.**
 - 예를 들어, ['That movie is famous movie', 'I like that actor', 'I don't like that actor'] 말뭉치를 BoW로 벡터화하면 다음과 같다.

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

- BoW를 이용해 벡터화하는 경우 모든 단어는 동일한 가중치를 갖는다. BoW 벡터를 활용해 영화 리뷰의 긍/부정 분류 모델을 만든다고 가정한다면 높은 성능을 얻기

는 어렵다.

- 'I like this movie'와 'I don't like this movie' 문장은 'don't'라는 단어가 문장 분류에 결정적인 역할을 하지만, 'don't'라는 단어가 자주 등장하지 않으므로 토큰화나 분류 모델 진행 시 해당 데이터가 무시될 수 있다.

단어 빈도 (Term Frequency, TF)

- 문서 내에서 특정 단어의 빈도수를 나타내는 값
 - Ex. 3개의 문서에서 'movie'라는 단어가 4번 등장한다면 해당 단어의 TF 값은 4가 된다. 이를 수식으로 나타내면 다음과 같다.

$$TF(t,d) = count(t,d)$$

- 앞선 BoW 벡터 표현 방법 과 같이 문서 내에서 등장한 빈도수를 계산하며, 해당 단어의 상대적 중요도를 측정하는데 사용된다.
- TF 값이 높을 수록 해당 단어가 특정 문서에서 중요한 역할을 한다고 생각할 수도 있지만, 단어 자체가 특정 문서 내에서 자주 사용되는 단어이므로 전문 용어나 관용어로 간주할 수 있다. TF는 단순히 단어의 등장 빈도수를 계산하기 때문에 문서의 길이가 길어질 수록 해당 단어의 TF 값도 높아질 수 있다.

문서 빈도 (Document Frequency, DF)

- 한 단어가 얼마나 많은 문서에 나타나는지를 의미한다. 특정 단어가 많은 문서에 나타나면 문서 집합에서 단어가 나타나는 횟수를 계산한다.
 - Ex. 3개의 문서에서 'movie'라는 단어가 4번 등장한다면 해당 단어의 DF 값은 3가 된다. 이를 수식으로 나타내면 다음과 같다.

$$DF(t,D) = count(t \in d: d \in D)$$

- DF는 단어가 몇 개의 문서에 등장하는지 계산하기 때문에 DF 값이 높으면 특정 단어가 많은 문서에서 등장한다고 볼 수 있다. 그 단어는 일반적으로 널리 사용되며, 중요도가 낮을 수 있다.
- 반대로 DF 값이 낮다면 특정 단어가 적은 수의 문서에만 등장한다는 뜻이다. 그러므로 특정한 문맥에서만 사용되는 단어일 가능성이 있으며, 중요도가 높을 수 있다.

역문서 빈도(Inverse Document Frequency, IDF)

- 전체 문서 수를 빈도로 나눈 다음에 로그를 취한 값을 말한다. 이는 문서 내에서 특정 단어가 얼마나 중요한 지를 나타낸다.
- 문서 빈도가 높을수록 해당 단어가 일반적이고 상대적으로 중요하지 않다는 의미가 된다. 그러므로 문서 빈도의 역수를 취하면 단어의 **빈도수가 적을수록 IDF 값이 커지게 보정**하는 역할을 한다.
- IDF는 분모의 DF 값에 1을 더한 값을 사용한다. 특정 단어가 한 번도 등장하지 않는다면 분모가 0이 되는 경우가 발생한다.

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

- 추가로 IDF는 로그를 취한다. 전체 문서 수를 빈도로 나눈 값을 사용한다면 너무 큰 값이 나올 수 있다. 10,000개의 문서에서 특정 단어가 1번만 등장한다면 IDF 값은 5,000이 된다. 이런 문제점을 방지하고자 로그를 취해 정교한 가중치를 얻는다.

TF-IDF

- 앞선 문서 빈도와 역문서 빈도를 곱한 값으로 사용한다. TF-IDF를 수식으로 표현하면 다음과 같다.

$$TF-IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

- 문서 내에 단어가 자주 등장하지만, 전체 문서 내에 해당 단어가 적게 등장한다면 TF-IDF 값은 커진다. 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어 등의 가중치는 낮아진다.
- `scikit-learn` 패키지를 사용해 TF-IDF를 계산한다.

TF-IDF 클래스

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer(
    input = 'content', # 입력될 데이터 형태를 의미한다. 'content': 문자열/바이트 형태,
    encoding = 'utf-8', # 바이트 혹은 입력값을 받을 경우 사용할 텍스트 인코딩 값
    lowercase = True,
    stop_words=None, # 분석에 도움이 되지 않는 의미 없는 단어들, 입력 받은 단어들은
    ngram_range=(1, 1), # N-gram range (min, Max). (1, 1): unigram, (1, 2): unigram, (2, 2): bigram
    max_df=1.0, # 전체 문서 중 일정 횟수 이상 등장한 단어를 불용어로 처리. 정수: count
```

```

min_df=1, # 전체 문서 중 일정 횟수 미만으로 등장한 단어를 불용어로 처리. max_df=
vocabulary=None, # 미리 구축한 단어 사전이 있다면 해당 단어 사전 사용
smooth_idf=True, # IDF 계산시 분모에 1을 더한다
)

```

6.2 TF-IDF 계산

```

corpus = [
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray()) # TF-IDF를 넘파이 배열로 변환 (문서 수) x (단어 수). 행
print(tfidf_vectorizer.vocabulary_) # vocabulary_: TF-IDF에 사용된 단어 사전

```

```

[[0.         0.         0.39687454 0.39687454 0.         0.79374908
  0.2344005 ]
 [0.61980538 0.         0.         0.         0.61980538 0.
  0.48133417]
 [0.4804584  0.63174505 0.         0.         0.4804584  0.
  0.37311881]]
{'that': 6, 'movie': 5, 'is': 3, 'famous': 2, 'like': 4, 'actor': 0, 'don': 1}

```

- 사이킷런의 TF-IDF 클래스는 `fit` 메서드를 통해 학습해야 데이터의 변화 또는 추론이 가능하다. `TF-IDF` 클래스를 통해 `tfidf_vectorizer` 객체를 불러오고, `fit` 메서드를 통해 준비된 말뭉치를 학습한다.
- 객체 학습이 완료되면 `transform` 메서드를 이용해 데이터 변환을 수행할 수 있다. `fit_transform` 메서드를 통해 학습과 변환을 동시에 수행할 수 있다.
- TF-IDF 에서 점수가 가장 높은 값을 세 개만 추려 색인으로 정리하면 `[[2, 3, 5], [0, 4, 61], [8, 1, 4]]` 가 된다. 이를 단어 사전과 매핑하면 `[[famous, is, movie], [actor, like, that], [actor, don, like]]` 가 된다.
 - 이를 통해 문서마다 중요한 단어만 추출할 수 있으며, 벡터값을 활용해 문서 내 핵심 단어를 추출할 수 있다.

- 하지만, 빈도 기반 벡터화는 문장 순서나 문맥은 고려하지 않기 때문에 문장 생성과 같이 순서가 중요한 작업에는 부적합하다. 또한, 벡터가 해당 문서 내의 중요도를 의미할 뿐, 벡터가 단어의 의미를 담고 있지는 않다.
 - ['나는 바나나를 먹었다', '그녀가 과일을 섭취한다', '고양이는 야옹 운다'] 를 벡터화 한다면 '나는 바나나를 먹었다' 와 '그녀가 과일을 섭취한다' 가 '고양이는 야옹 운다' 보다 더 가까운 의미를 가지고 있지만, 세 문장 간의 유사도는 동일하다.
-